24/03/26

Ex. No: 8 Producer Consumer Using Semaphores

Aim:

To implement the Producer-Consumer problem using semaphores in C for process synchronization.

Source Code:

```
#include <stdio.h>
#include < stdlib.h >
int mutex=1 , full=0. empty=5 , x=0;
int wait (int s)
{
  return --s;
}
int signal (int s)
{
  return ++s;
}
void producer()
{
  mutex = wait (mutex);
  empty = wait (empty);
  full = signal ( full);
  x++;
  printf ("\n Producer produces item %d ", x);
  mutex= signal (mutex);
}
void consumer()
{
  mutex = wait (mutex);
  full = wait (full);
```

```c
empty = signal (empty);
printf("\n Consumer consumes item %d", x);
x--;
mutex = signal (mutex);
}
int main()
{
  int n;
  printf("\n ----Producer Consumer Problem ----");
  printf("\n 1. Producer");
  printf("\n 2. Consumer");
  printf("\n 3. Exit");
  while (1)
  {
    printf("\n Enter your choice:");
    scanf("%d", &n);
    switch (n)
    {
     case 1:
        if (mutex==1 && empty!=0)
        {
          producer();
        }
        else
        {
           printf("\n Buffer is full! Producer cannot produce.");
        }
        break;
     case 2:
        if (mutex==1 && full!=0)
```

```
{
  consumer();
}
else
{
  printf("\n Buffer is empty !! Consumer cannot consume. ");
}
break;
case 3:
   printf("\n Exiting program..... ");
   printf(" \n Final buffer items produced: %d \n", x);
   exit (0);
default:
   printf("\n Invalid choice. Try again. ");
}
}
return 0;
}
```

Output:

```
---- Producer Consumer Problem ----
1. Producer
2. Consumer
3. Exit
Enter your choice: 1

Producer produces item 1
Enter your choice: 1

Producer produces item 2
Enter your choice: 1
```

Producer produces item 3
Enter your choice : 2

Consumer consumes item 3
Enter your choice : 2

Consumer consumes item 2
Enter your choice : 2

Consumer consumes item 1
Enter your choice : 2

Buffer is empty!! Consumer cannot consume.
Enter your choice : 3

Exiting program.....
Final buffer items produced : 0

Result:

The program successfully controls producer and consumer operations using semaphores, preventing buffer overflow and underflow.